Assignment 2: Stochastic Multi-Elevator Passenger

Bar-Ilan CS 89-570
Introduction to Artificial Intelligence

May 18, 2026

1 Introduction

In this exercise you will continue working with the *Multi-Elevator* domain from Assignment 1. The physical world is exactly the same: several elevators move people inside a building, respecting their reachable-floor sets and weight capacities.[1] **Now, however, our elevators and passengers are unreliable: every action has a probability of failing.**

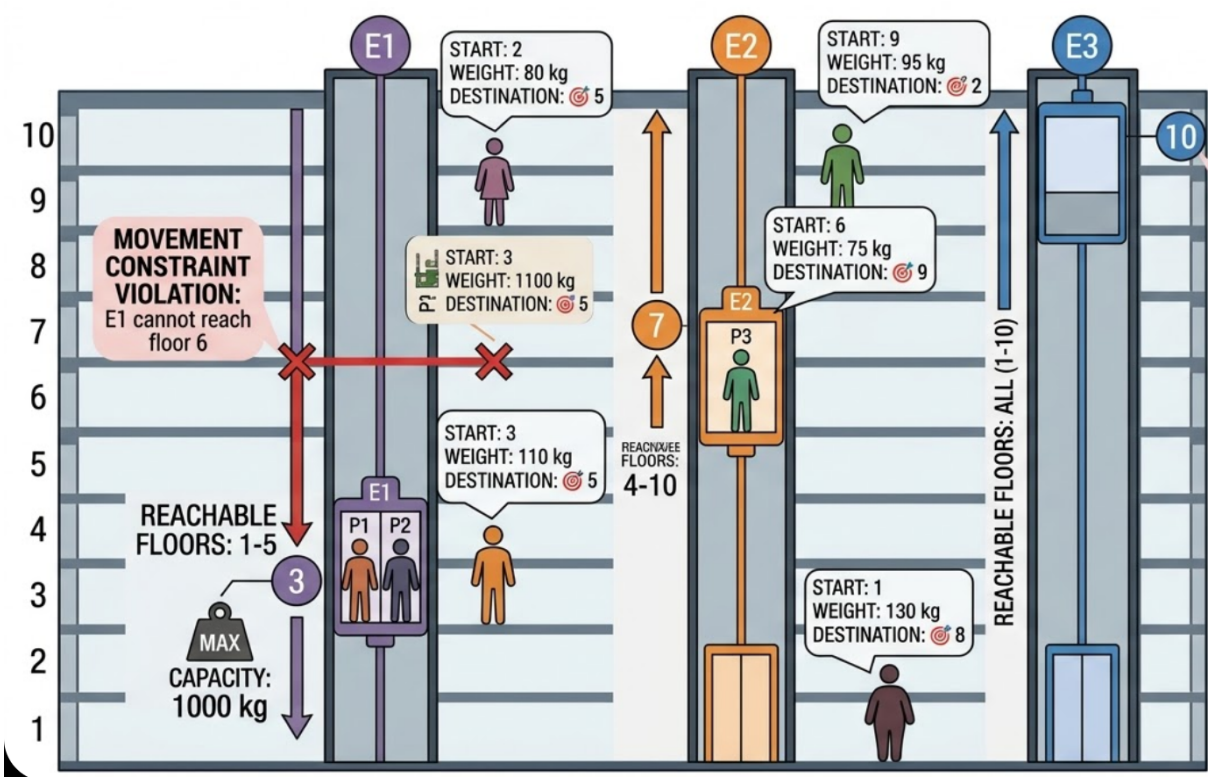


Figure 1: The original *Multi-Elevator* domain from Assignment 1.

[1] See the description in Assignment 1 for the full deterministic version of the domain.

In Assignment 1 you solved a *deterministic* planning / path-finding problem using A*. Here we move to a *stochastic* setting: the same world is now modelled as a Markov Decision Process (MDP).

The key differences are:

- Elevators are **defective**: each elevator has a probability of successfully executing the MOVE action it was told to perform, and with the remaining probability it "misbehaves" and ends up on a different floor.
- People are also **unreliable**: each person has a probability of successfully ENTERing / EXITing when an action targets them. With the remaining probability the action has no effect.
- Successful deliveries give **stochastic rewards**: every time a person successfully exits an elevator *on their destination floor*, a reward is sampled uniformly at random from a person-specific list of possible rewards.

Your task is to implement a controller function that, given the current state of the task, chooses the next action. The goal is to **maximize the accumulated reward** over the episode. The exact reward definition and problem dynamics are described below.

2 Problem Description

2.1 Reminder: Deterministic core (Assignment 1)

We work in the same building as in Assignment 1:

- Floors are numbered $0, 1, \ldots,$ `height`. Floor 0 is the ground floor.
- Each elevator has a unique ID, a current floor, a set of floors it can reach, and a maximum weight capacity.
- Each person has a unique ID, a starting floor, a weight, and a destination floor. A person may either stand on some floor or be inside exactly one elevator.

Elevators and people interact through the three basic actions of Assignment 1:

- MOVE(ELEVATOR_ID, TARGET_FLOOR): move an elevator from its current floor to another floor in its reachable set.
- ENTER(PERSON_ID, ELEVATOR_ID): move a person from their current floor into an elevator on the same floor. Legal only if the person is not already inside an elevator and the total weight inside the elevator does not exceed its maximum capacity after entering.
- EXIT(PERSON_ID, ELEVATOR_ID): move a person out of an elevator to the elevator's current floor. Legal only if the person is currently inside that elevator.

In Assignment 1 the task was purely deterministic: you just needed to find an optimal sequence of actions that brings every person to their destination floor.

2.2 New stochastic extensions (Assignment 2)

In this assignment we keep the same world, but turn it into a stochastic MDP by changing the behaviour of elevators, people, and rewards:

- **Defective elevators.** Each elevator e has a parameter `elevator_chosen_action_prob[e]` in the problem. This value is the probability that elevator e *successfully* executes the MOVE action it was told to perform. With the remaining probability the MOVE fails and the elevator ends up on a different floor (described below).
- **Unreliable people.** Each person p has a parameter `person_chosen_action_prob[p]` in the problem. This value is the probability that an ENTER or EXIT action targeting p *succeeds*. With the remaining probability the action has no effect (the person stays where they were).
- **Random delivery rewards.** Each person p has a list of possible rewards in `persons_reward[p]`. Whenever person p *successfully* exits an elevator *on their destination floor*, a reward is sampled uniformly at random from `persons_reward[p]` and added to the total reward; person p is then considered delivered and removed from the state.
- **Goal reward.** When *every* person has been delivered, the episode is considered completed and an additional deterministic `goal_reward` is given. The state is then reset to the initial state and the episode continues; the horizon keeps running.
- **Horizon.** The maximum number of steps the controller has to accumulate reward.

Unlike Assignment 1, you do not provide a sequence of actions in advance. Instead, at each step the controller receives the current state of the task and must decide which action to execute next, trying to maximise the total expected reward.

2.3 Actions and their outcomes

For each chosen action, the engine performs the following steps:

1. **Legality check.** First, the task checks that the chosen action is legal in the current state:
   - MOVE(E, F) is legal only if f belongs to the reachable-floor set of e.
   - ENTER(P, E) is legal only if person p and elevator e are on the same floor, p is not already inside any elevator, and the total weight of people inside e after entering does not exceed the elevator's maximum capacity.
   - EXIT(P, E) is legal only if person p is currently inside elevator e.

   If the action is illegal, the engine raises an error – something that should never occur with a correct controller.
2. **Success vs. failure.** If the action is legal, the engine flips a biased coin (using the appropriate probability for that elevator or that person):
   - With the success probability the action *succeeds* and its intended effect is applied.
   - With the remaining probability the action *fails*, leading to a different outcome.

We now specify the exact effect of each action in both cases.

move(e, f). Let F_e denote the set of floors reachable by elevator e, and let f_0 be the current floor of e.

- **On success** (with probability `elevator_chosen_action_prob[e]`): elevator e moves from f_0 to f.
- **On failure** (with the remaining probability): the new floor of e is chosen uniformly at random from
$$\{f_0\} \cup (F_e \setminus \{f\}),$$
i.e. *stay in place* or *go to any reachable floor other than the intended target.*

enter(p, e).

- **On success** (with probability `person_chosen_action_prob[p]`): person p enters elevator e; the carried-weight inside e is increased by the weight of p.
- **On failure** (with the remaining probability): nothing changes – person p stays on the floor, elevator e is unaffected, and no reward is obtained.

exit(p, e).

- **On success** (with probability `person_chosen_action_prob[p]`): person p exits e onto e's current floor; the carried-weight inside e is decreased by the weight of p. Then:
  - if e's current floor equals the destination floor of p, a reward is sampled uniformly at random from `persons_reward[p]` and added to the total reward; person p is removed from the state (delivered);
  - otherwise, no reward is obtained and p now stands on e's current floor.
- **On failure** (with the remaining probability): nothing changes – person p stays inside elevator e and no reward is obtained.

reset. At any step, the controller may choose to reset the state to the initial state. This action always succeeds (probability 1), gives no reward, and costs 1 step. If your step budget was x before applying RESET, it will be $x - 1$ after.

Episode termination and goal reward. Whenever every person has been delivered, the controller receives the additional `goal_reward`, the episode is considered finished, and the task is reset to the initial state. The horizon keeps running across episodes. **Note that every action costs 1 step (reset included).**

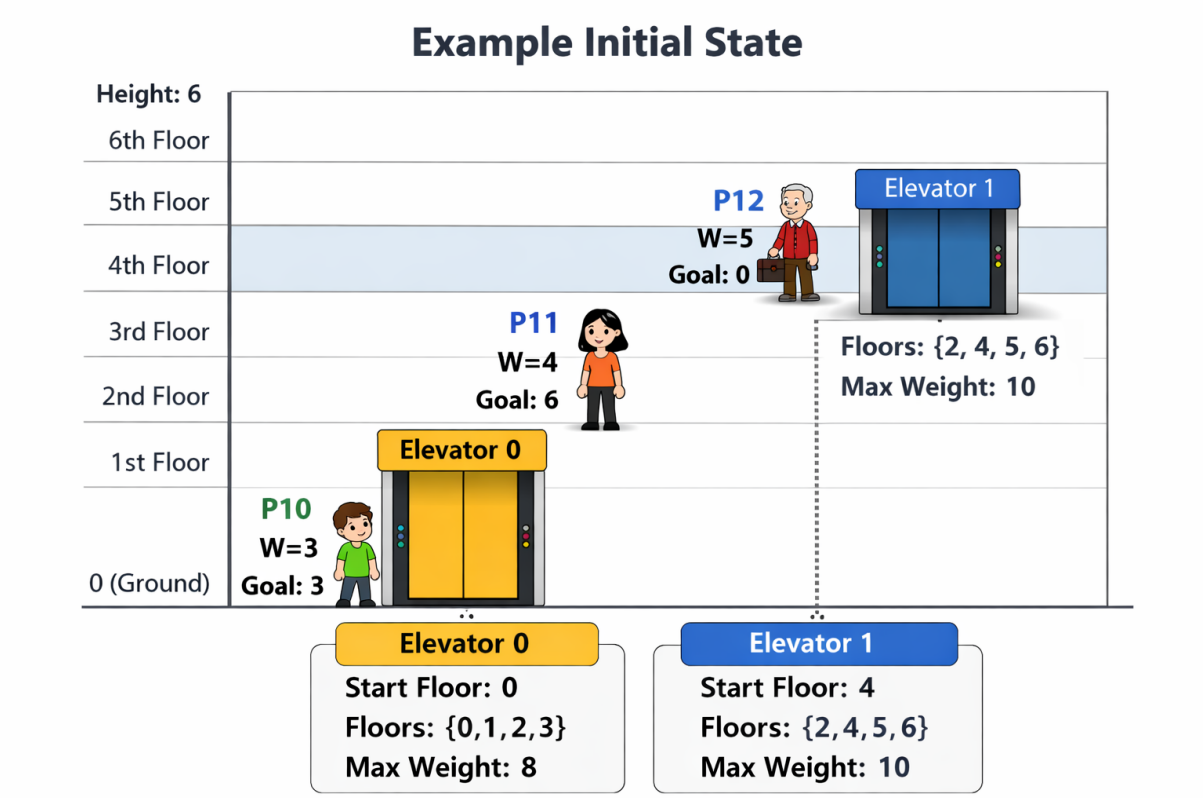
3 Problem Specification

3.1 Problem dictionary

The problem is specified as a Python dictionary `problem`, very similar to Assignment 1, with the following entries:

1. **height** – an integer. The building floors are $\{0, 1, \ldots, \texttt{height}\}$.
2. **Elevators** – a dictionary whose keys are elevator IDs. The value of elevator ID e is a tuple
$$(f, F, w_{\max}),$$
where f is the current floor of the elevator, F is the set of floors this elevator can reach, and $w_{\max}$ is its maximum weight capacity.
3. **Persons** – a dictionary whose keys are person IDs. The value of person ID p is a tuple
$$(f_{\text{start}}, w, f_{\text{goal}}),$$
where f_{start} is the person's starting floor, w is the person's weight, and f_{goal} is the destination floor.
4. **elevator_chosen_action_prob** – a dictionary. Each key is an elevator ID, and the value is a number in $[0, 1]$ giving the probability that this elevator successfully executes the MOVE action it chooses.
5. **person_chosen_action_prob** – a dictionary. Each key is a person ID, and the value is a number in $[0, 1]$ giving the probability that an ENTER or EXIT action targeting this person succeeds.
6. **persons_reward** – a dictionary. Each key is a person ID p, and the value is a list of possible rewards for delivering p. Whenever person p is successfully delivered (EXIT on p's destination floor), the actual reward is sampled *uniformly at random* from this list.
7. **goal_reward** – a number giving the reward received when *every* person has been delivered (i.e., when the global goal is achieved).
8. **horizon** – the number of steps the controller has to accumulate reward.

3.1.1 Example




```
problem = {
    "height": 6,
    "Elevators": {
        0: (0, (0, 1, 2, 3), 8),
        1: (4, (2, 4, 5, 6), 10)
    },
    "Persons": {
        10: (0, 3, 3),
        11: (2, 4, 6),
        12: (4, 5, 0)
    },
    "elevator_chosen_action_prob": {0: 0.8, 1: 0.7},
    "person_chosen_action_prob":   {10: 0.9, 11: 0.6, 12: 0.85},
    "persons_reward": {
        10: [2, 3, 6, 10],
        11: [1, 5, 6, 10],
        12: [3, 4, 8, 12]
    },
    "goal_reward": 30,
    "horizon": 100
}
```

3.2 Goal description

Unlike Assignment 1, where the goal was simply to deliver every person, here we care about *maximizing reward*. Formally, the goal of your controller is to choose actions that **maximize the expected total reward** obtained during the episode, where the reward comes from:

- stochastic rewards when persons are successfully delivered to their destination floors, and
- a deterministic `goal_reward` once every person has been delivered. After receiving the `goal_reward`, the state is reset to the initial state, and the remaining step budget continues to be spent.

Your implementation will not construct a full plan in advance, but will decide, at each step, which action to take based on the current state.

4 State Representation

In this assignment, the state representation is fixed and you must not change it:

`(elevators_t, persons_t, total_persons_remaining)`

Where:

- `elevators_t` is a tuple of elevators. Each elevator is represented as $(eid, f, w_{\text{load}})$, where eid is the elevator's ID, f is the elevator's current floor, and w_{load} is the total weight currently inside the elevator.
  Note: you can obtain an elevator's maximum capacity and reachable set using provided functions.
- `persons_t` is a tuple of persons that have *not yet been delivered*. Each person is represented as (pid, loc), where pid is the person's ID and loc encodes the person's current location:
  - $loc = (\texttt{'floor'}, f)$ – the person is standing on floor f;
  - $loc = (\texttt{'in'}, eid)$ – the person is inside elevator eid.

  Note: a person's starting floor, weight, destination, and reward list are static and can be obtained using provided functions.
- `total_persons_remaining` is the number of persons that have not yet been delivered.

5 Engine API

Your controller interacts with the engine through a single object of type `GameAPI`. The full set of methods you may call is listed below; any access beyond this list is forbidden (see the engine-access policy at the end of this section). Each method returns an *immutable copy* of internal data when relevant, so mutating a returned object has no effect on the engine.

- Returns the current state tuple (see Section 4):

```
def get_current_state(self):
    return self._state
```

- Returns the initial state tuple — the state the task is reset to after a successful goal episode or after RESET:

```
def get_initial_state(self):
    return self._initial_state
```

- Returns whether the task is finished (e.g., when `steps == max_steps`):

```
def get_done(self):
    return self._done
```

- Returns the number of steps performed so far:

```
def get_current_steps(self):
    return self._steps
```

- Returns the horizon of the problem (maximum number of steps):

```
def get_max_steps(self):
    return self._max_steps
```

- Returns the current accumulated reward obtained from the actions taken so far:

```
def get_current_reward(self):
    return self._reward
```

- Returns the deterministic reward awarded when every person has been delivered:

```
def get_goal_reward(self):
    return self._goal_reward
```

- Returns the success probability of elevator e's MOVE actions:

```
def get_elevator_action_prob(self, e):
    return self._elevator_action_prob[e]
```

- Returns the success probability of ENTER/EXIT actions targeting person p:

```
def get_person_action_prob(self, p):
    return self._person_action_prob[p]
```

- Returns the weight of person p:

```
def get_person_weight(self, p):
    return self._person_weight[p]
```

- Returns the destination floor of person p:

```
def get_person_goal(self, p):
    return self._person_goal[p]
```

- Returns the (immutable) tuple of possible rewards for delivering person p; one of these values is sampled uniformly at random on a successful delivery:

```
def get_person_reward(self, p):
    return tuple(self._persons_reward[p])
```

- Returns a fresh dictionary of elevator capacities (`eid : max_weight`):

```
def get_capacities(self):
    return dict(self._capacities)
```

- Returns a fresh dictionary of elevator reachable-floor sets (`eid : frozenset`):

```
def get_reachable(self):
    return dict(self._reachable)
```

Engine access policy. Your controller may interact with the engine **only** through the getter methods listed above and `submit_next_action`. The following are strictly forbidden and will result in a grade of **zero** for this assignment:

- Accessing any attribute of the engine object whose name starts with an underscore (single `_` or double `__`).
- Modifying any data returned by a getter and assuming the engine reflects the change.
- Re-seeding, replacing, or otherwise manipulating the engine's random number generator (including `np.random.seed`, `np.random.default_rng`, or shadowing the `numpy` module).
- Modifying probabilities, rewards, capacities, reachable sets, or any other problem parameter at runtime.
- Importing `ext_elev` internals beyond the documented API, monkey-patching its classes or functions, or using `vars()`, `__dict__`, `gc`, `ctypes`, or similar introspection facilities to inspect or modify engine state.

Submissions will be inspected for these patterns. No warning is given — the first occurrence is sufficient for a zero.

6 What You Must Implement

We provide a starter codebase. Most of the infrastructure is already implemented. Your job is to complete the methods so that the checker can run instances.

Files and required functions (in `ex2.py`)

You *must* implement the following (you may add helpers as you see fit):

1. **`choose_next_action(state) -> str`**
   - **Input:** a state tuple `state` = (`elevators_t`, `persons_t`, `total_persons_remaining`).
   - **Output:** a *single* legal action encoded as a string, in the same format used in Assignment 1:
     - MOVE(E, F): `MOVE{e,f}`
     - ENTER(P, E): `ENTER{p,e}`
     - EXIT(P, E): `EXIT{p,e}`
     - RESET: `RESET` (with no arguments)
   - **Possible actions (depending on legality):** MOVE, ENTER, EXIT, RESET.

7 Scoring & Evaluation

- There will be **8 problem instances**.
- Each instance will be evaluated using **30 different random seeds**. For each seed, we run the instance once, record the total reward, and then compute the instance score as the average reward across all tested seeds (sum of rewards divided by the number of seeds).
- Time limit for each instance and seed is: $20 + 0.5 \cdot$ `horizon`.
- To receive a passing grade (60):
  - For each problem in the test set provided with the starter code, **your average reward must be higher than that of a random policy**.
  - There is a **validation check in `ext_elev.py`**. Make sure that every action you return from `choose_next_action` passes this check.
- To receive a grade higher than 80, your code must surpass the performance of our baseline, which we will publish in the next days.
- The last 20% of the grade will be determined by a competition.

8 Attached files

The starter code includes the following files:

1. **`ex2.py`** – *the only file you submit, and the only file that will be graded.* Implement `__init__`, `choose_next_action`, and set your `id` variable. Do not rename or move this file.
2. **`ex2_check.py`** – a local *self-checker*. Use it to run your code on the provided instances. You may add additional problems here for your own testing, but the grading system will *not* use your added problems. Changes to this file do not affect grading.
3. **`ext_elev.py`** – the engine that runs your action-selection function on the current state. You interact with it only through the `GameAPI` object documented in Section 5. You do not submit this file; do not modify it.
4. **`ex2_random.py`** – a *random baseline controller*. Picks uniformly at random from the legal actions at each step. This is the policy you must beat to receive a passing grade.
5. **`ex2_gui.py`** – an optional *visualizer* built with `pygame`. Renders the building, elevators, and persons step-by-step so you can watch your controller play. Not used for grading.

Running the code

All commands are run from the directory containing the starter files.

Self-checker. Runs your controller across all provided instances and prints average rewards:

```
python ex2_check.py
```

Random baseline. To measure the random policy on the same instances (useful for sanity-checking the bar you need to clear), edit `ex2_check.py` and replace `import ex2` with `import ex2_random as ex2`, then run it as above.

Visualizer. Requires `pygame` (`pip install pygame`). Launch with:

```
python ex2_gui.py
```

A window opens showing the building. Controls:

- `SPACE` – advance one step
- `1` – run one full episode
- `0` – run all episodes back-to-back
- `ESC` – quit

By default the GUI drives the engine with `ex2_random`. To watch your own controller, change the top-of-file import `import ex2_random as student` to `import ex2 as student`. You can also swap the `problem` dict at the top of the file to visualize a different instance.

9 Rules & Submission

- **Edit only `ex2.py`.** Do not modify or submit any other file.
- Rename `ex2.py` to `ex2_id.py`, where `id` is your ID number. For example: `ex2_000000000.py`.
- Submit the renamed file using the Bar-Ilan `submit` system: `https://submit.cs.biu.ac.il/cgi-bin/welcome.cgi`
- **Individual work.** Discussing ideas is allowed, but *sharing or copying code is prohibited.*
- **Use of AI / online sources.** You may use them, but you *must* clearly state *at the top of the document* which LLMs or online sources you used and how you used them (for example: writing parts of the code, suggesting specific actions, providing ideas, or anything else). Failure to disclose this information will result in a grade reduction.
- **Academic integrity.** We take plagiarism seriously, please do not make us deal with it.
- **Questions.** Ask during office hours or in the assignment forum only.
- **Extensions.** No extensions will be granted except for documented exceptional circumstances via email.
- **Deadline: 11/6/2026**.

Rules (summary).

- Unit action cost (each action, RESET included, costs one step).
- At each step, exactly one action is executed.
- ENTER requires the person and the elevator to be on the same floor, and weight capacity must be respected.
- EXIT requires the person to be currently inside that elevator.
- MOVE requires the target floor to be in the elevator's reachable set.
- Each person starts standing on their starting floor with no elevator assignment.